Internal Working of Heap

What is Heap?

The heap is a region of memory used for dynamic memory allocation in Java. It's where all objects and instance variables are stored during program execution. The heap is managed by the Java Virtual Machine (JVM) and is shared among all threads in a Java application.

Key Characteristics of Heap:

- **Dynamic allocation**: Objects are created at runtime
- **Garbage collected**: Unused objects are automatically removed
- **Shared memory**: All threads can access heap memory
- **Slower access**: Compared to stack memory due to dynamic nature

Heap Structure:

The heap is divided into different generations to optimize garbage collection:

1. **Young Generation**
   - Eden Space: Where new objects are created
   - Survivor Spaces (S0, S1): Where objects survive initial garbage collection
2. **Old Generation (Tenured Space)**
   - Where long-lived objects are stored after surviving multiple garbage collections
3. **Metaspace** (Java 8+)
   - Stores class metadata and method information

How Heap Works

Object Creation Process

When you create an object using the `new` keyword, the following happens:

```java
// Example of object creation
String name = new String("Hello");
Person person = new Person("John", 25);
```

1. **Memory Allocation**: JVM allocates memory in the Eden space of Young Generation

2. **Constructor Execution**: Object constructor is called to initialize the object
3. **Reference Assignment**: A reference to the object is created and stored in the stack

Memory Allocation Example

```java
public class HeapExample {
    public static void main(String[] args) {
        // Objects created in heap
        String str1 = new String("Java");
        String str2 = new String("Programming");

        // Array created in heap
        int[] numbers = new int[5];

        // Custom object created in heap
        Student student = new Student("Alice", 20);
    }
}

class Student {
    String name;    // Instance variable stored in heap
    int age;        // Instance variable stored in heap

    public Student(String name, int age) {
        this.name = name;
        this.age = age;
    }
}
```

Garbage Collection Process

The heap uses automatic memory management through garbage collection:

1. **Minor GC**: Cleans up Young Generation
   - Objects in Eden space are moved to Survivor space
   - Objects that survive multiple cycles are promoted to Old Generation
2. **Major GC**: Cleans up Old Generation
   - Removes long-lived objects that are no longer referenced
3. **Full GC**: Cleans up entire heap
   - Most expensive operation, affects application performance

Heap Memory Allocation Example

```java
public class HeapMemoryDemo {
    // Class variables stored in Metaspace
    static int classVariable = 100;

    // Instance variables stored in heap
    String instanceVariable = "Heap Memory";

    public void demonstrateHeap() {
        // Local object reference stored in stack
        // Actual object stored in heap
        StringBuilder sb = new StringBuilder();
        sb.append("This string is in heap");

        // Array object stored in heap
        String[] array = new String[3];
        array[0] = "First";
        array[1] = "Second";
        array[2] = "Third";

        // Each string object stored in heap
        for(String item : array) {
            System.out.println(item);
        }
    }
}
```

Object Lifecycle in Heap

```java
public class ObjectLifecycle {
    public static void main(String[] args) {
        // Step 1: Object created in Eden space
        MyObject obj1 = new MyObject("Object1");

        // Step 2: More objects created
        MyObject obj2 = new MyObject("Object2");
        MyObject obj3 = new MyObject("Object3");

        // Step 3: Remove reference (object becomes eligible for GC)
        obj1 = null;

        // Step 4: Suggest garbage collection
        System.gc(); // Not guaranteed to run immediately

        // Step 5: Create new objects
        for(int i = 0; i < 1000; i++) {
            MyObject temp = new MyObject("Temp" + i);
            // temp goes out of scope, eligible for GC
        }
    }
}

class MyObject {
    private String name;

    public MyObject(String name) {
        this.name = name;
        System.out.println("Object created: " + name);
    }

    // Finalizer method (called before GC)
    protected void finalize() {
        System.out.println("Object destroyed: " + name);
    }
}
```

Heap vs Stack Comparison

```java
public class HeapVsStack {
    int instanceVar = 10;  // Stored in heap

    public void method() {
        int localVar = 20;     // Stored in stack
        String str = "Hello";  // Reference in stack, object in heap

        // Array reference in stack, array object in heap
        int[] arr = {1, 2, 3, 4, 5};
    }
}
```

Memory Leaks and Heap Management

```java
import java.util.*;

public class HeapManagement {
    // Potential memory leak - static collection grows indefinitely
    private static List<String> staticList = new ArrayList<>();

    public void addToStaticList(String item) {
        staticList.add(item);
        // Objects in staticList won't be garbage collected
    }

    public void properMemoryUsage() {
        // Local collection - eligible for GC when method ends
        List<String> localList = new ArrayList<>();
        localList.add("Temporary data");

        // Process data
        for(String item : localList) {
            System.out.println(item);
        }

        // localList becomes eligible for GC when method ends
    }
}
```

Quiz Questions

1. Where are objects stored in Java memory? a) Stack b) Heap c) Method Area d) PC Register

Answer: b) Heap

2. Which part of heap is used for newly created objects? a) Old Generation b) Survivor Space c) Eden Space d) Metaspace

Answer: c) Eden Space

3. What happens when an object has no references pointing to it? a) It remains in memory forever b) It becomes eligible for garbage collection c) It is immediately deleted d) It moves to stack memory

Answer: b) It becomes eligible for garbage collection

4. Which type of garbage collection cleans up the Young Generation? a) Major GC b) Full GC c) Minor GC d) Complete GC

Answer: c) Minor GC

5. What is stored in the Metaspace (Java 8+)? a) Object instances b) Local variables c) Class metadata and method information d) Primitive data types

Answer: c) Class metadata and method information

Practice Coding Questions

1. Create a class called `MemoryDemo` with instance variables of different data types. Create multiple objects of this class and demonstrate how they are stored in heap memory.

2. Write a program that creates a large number of objects in a loop and demonstrates the garbage collection process. Include a finalizer method to track when objects are destroyed.

3. Implement a class that shows the difference between shallow copy and deep copy in terms of heap memory allocation.

4. Create a program that demonstrates potential memory leaks by storing objects in static collections. Then modify it to avoid memory leaks.

5. Write a program that creates nested objects (objects containing other objects) and explain how they are stored in heap memory with proper memory allocation diagrams in comments.